
Checking Quran Recitation from Noisy ASR Transcripts: a Two-Task Competition

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Every Quran-memorization app that listens to a learner has to answer the same
2 question from a noisy ASR transcript: which ayahs were recited, where each one
3 starts and ends, and whether the learner should be corrected, all without flagging the
4 breath repeats, restarts and stutters that make up most real recitation. We propose a
5 competition on this checking problem and release 258 human-reviewed transcripts
6 of production recordings (1,267 gold ayah chunks) with two fixed scorecards,
7 a recording-level held-out split with a zero oracle floor, a 10% review sample,
8 checksums, and three evaluated baselines, one being a deployed app’s hand-built
9 pipeline.

10 1 Why this task, and how we frame it

11 Hundreds of millions of Muslims memorize Quran, almost always with a listener who corrects them.
12 Apps that stand in for the listener transcribe the recitation with commercial ASR and then have to
13 check noisy text: isti’adhah and basmala, ASR errors, repeats after catching a breath, full restarts,
14 stutters, self-corrections, skipped words, abandoned ayahs. A plain word-error diff calls all of that a
15 mistake. In our production data 87% of ayah chunks are clean, and an app that keeps flagging benign
16 behaviour loses a learner’s trust faster than one that misses an occasional error. Public Quran speech
17 resources target ASR of professional read recitation; as far as we know, none labels checking on real
18 memorization attempts.

19 We frame it as two tasks on one dataset. In **Task A (detect and split)** the input is one raw transcript
20 and the output is the set of ayah ids recited plus a segmentation of the words by ayah, or *abstain*
21 when the input is not Quran; it is already frozen (v1.1). In **Task B (within-ayah mistake events)** the
22 input is the transcript words of one ayah (a gold Task A chunk), the reference text, and the chunk’s
23 position in the recording. The output is a list of events, each with a type (restart_repeat, full_repeat,
24 stutter, self_correction, substitution, omission, insertion, word_order, truncation, asr_garble) and a
25 verdict: *benign*, *corrected*, *mistake*, or *uncertain*. The last verdict exists because text alone cannot
26 always tell ASR error from recitation error, and a system should be allowed to say so. Task B ships
27 as a labeled pilot now (Section 4).

28 2 Dataset

29 **Source and consent.** The 258 recordings come from adult users of a Quran-memorization chat bot,
30 under terms of service that disclose review for service improvement. A commercial ASR model
31 produced the transcripts, and a human reviewed every row (ayah assignment, split, confidence) after
32 LLM pre-labeling. 254 rows are Quranic, covering 604 distinct ayahs from 45 surahs, with 1–46
33 ayahs and 2–261 words (median 40) per recording; 4 are non-Quranic negatives. We release text only,
34 replace learner and recording identifiers with surrogate ids, and checked the negatives by hand; as the

Quran is a closed public canon, the transcripts carry essentially no personal information. **Split.** A deterministic stratified 60/40 split by recording (151/107 rows; 711/556 chunks), never by chunk, so one reciter’s ASR quirks cannot leak. The negatives split 2/2, and known-ambiguous rows are pinned to train so the held-out oracle floor is exactly 0. Task B uses the same split and ids.

Task B labels. The schema is frozen: verbatim hypothesis and reference words, token spans derived by script, a verdict, and a note. Three rules keep labels reproducible: a dropped word-final letter is *uncertain*; a truncated ayah is benign when it is the last in the recording (the learner simply stopped) and a mistake anywhere else; a slip inside an abandoned first attempt is absorbed by the repeat that follows. The pilot covers 99 chunks from 20 recordings, 86 of them clean, with 17 events (11 substitutions, 3 omissions, 2 truncations, 1 full repeat), proposed by an LLM and marked human-pending until reviewed. A naive diff flags only 21% of all 1,267 chunks; the rest match the reference exactly after canonicalization, so finishing the labels means reviewing about 270 chunks.

3 Metrics and baselines

Both scorecards are fixed scripts; lower is better; each component lies in $[0, 1]$ and is printed next to the scalar, and an empty system scores 2.0. For Task A, `research_score` = detection err + split err + abstention err (exact match on the set of ayah ids; word-assignment accuracy after a metric-only orthographic canonicalization; abstention on the negatives). For Task B, `study3_score` = $2 \cdot \text{miss} + \text{false_alarm} + \text{benign} + \text{clean}$. A prediction matches a gold event when their token spans overlap. *miss* is the share of gold mistakes not flagged (an *uncertain* prediction on one costs half); *false_alarm* is the share of flags that touch no gold mistake or uncertain event; *benign* is the share of gold benign or corrected events flagged as mistakes; *clean* is the share of clean chunks that received any flag, which gets its own term because clean chunks are the majority class. Adjacent same-type predictions are merged so word- and phrase-level detectors score alike; type agreement and the uncertain rate are reported, not summed.

Task	Baseline	train	test	test components	
A	empty stub	2.000	2.000	det 1, split 1	abstain 0
A	incumbent production pipeline (hand-built)	0.785 [†]	0.760	det 0.17, split 0.09	abstain 0.50
A	coding-agent loop, best of 6 runs [†]	0.008 [†]	0.079	det 0.06, split 0.02	abstain 0
B	B0 empty stub	2.000	2.000	miss 1	
B	B1 naive diff (every edit is a mistake)	1.358	1.111	benign 1	false_alarm 0.11
B	B2 incumbent cleaner + DP alignment	0.167	1.111	benign 1	false_alarm 0.11

Table 1: Baselines. [†]Full 258-row set. Task B is scored on the pilot labels (47 train / 52 test chunks); both existing designs flag the held-out full repeat as a mistake (*benign* = 1).

On Task A, six autonomous runs of two coding agents in a Karpathy-style fixed-metric loop all matched or beat the incumbent held-out, the best by $10 \times$ ¹; without a held-out split the same loop memorized rows, which is why the competition ends on a hidden test set.

4 Competition plan, feasibility and ethics

The competition launches with Task A frozen and Task B fully human-reviewed on the 258 recordings (about 270 chunks), runs a public leaderboard on the released test split, and ranks on a hidden test set of new recordings drawn from more than 20k further production recordings of the same app, labeled with the same protocol, user-reported false flags over-sampled as adversarial gold, and released afterwards. A submission is one Python file per contract and scoring takes under a second, so an agent track (autonomous loops, logged per experiment) runs beside the human track.

Harness, data, checksums and the 10% sample (26 recordings, both tasks) are attached, and the baselines run offline. The data is text only, de-identified, from adult users under ToS-disclosed review, and licensed CC BY 4.0. The task is corrective feedback, not surveillance, and *uncertain* is a first-class output so systems are not pushed to over-claim. Entry cost is low and every Quran-learning app faces this problem; we expect entrants from Arabic NLP, speech and agent groups.

¹Autoresearch with coding agents: generalizers and metric-maximizers on Quran recitation data, AIST 2026.